

Respuestas Parte 3 y Parte 4

Prueba Técnica Junior Full Stack

LinkedIn: [Jhonatan David Diaz Alvarez](#)

Repositorios:

https://github.com/JhonaLegitimo/PT_Backend_Certika

https://github.com/JhonaLegitimo/PT_Frontend_Certika

Parte 3 - AWS (Conceptos Básicos)

1. ¿Dónde desplegaría una aplicación React o Next.js?

Para una aplicación React la desplegaría en Amazon S3 usando hosting estático y, si se necesita mejor rendimiento, la conectaría con CloudFront. En el caso de Next.js, si la aplicación usa renderizado del lado del servidor o rutas dinámicas, la desplegaría en un servicio que soporte Node.js, como AWS Amplify, Elastic Beanstalk o una instancia EC2. Para este proyecto, como es una interfaz que consume una API REST, buscaría una opción simple que permita servir el frontend y configurar la URL del backend mediante variables de entorno.

2. ¿Dónde desplegaría una API Node.js?

La API Node.js la desplegaría en un entorno donde pueda ejecutar el servidor Express de forma continua. Una opción sería Amazon EC2, configurando Node.js, las variables de entorno y un proceso para mantener la API activa. También podría usarse Elastic Beanstalk, porque simplifica parte de la configuración del servidor. En este caso, el backend es una API REST con Express, CORS y variables de entorno, así que lo importante sería exponerla por HTTPS y permitir que el frontend pueda consumir sus endpoints.

3. ¿Qué servicio utilizaría para almacenar archivos?

Utilizaría Amazon S3. Es el servicio más adecuado cuando se necesita guardar archivos como imágenes, documentos, exportaciones o respaldos. La API podría subir los archivos a S3 y guardar en la base de datos solamente la información necesaria para consultarlos después, por ejemplo el nombre del archivo o la URL.

4. ¿Qué es Amazon S3?

Amazon S3 es un servicio de almacenamiento de objetos. Sirve para guardar archivos de forma escalable, segura y durable. En vez de manejar los archivos dentro del mismo servidor de la aplicación, se pueden guardar en buckets de S3 y controlar quién puede leerlos o escribirlos mediante permisos.

5. ¿Qué es Amazon EC2?

Amazon EC2 es un servicio que permite crear servidores virtuales en AWS. En una instancia EC2 se puede instalar Node.js, clonar el proyecto backend, configurar variables de entorno y ejecutar la API. Es flexible porque da control sobre el sistema operativo y la configuración, aunque también requiere más administración que otros servicios más automatizados.

6. ¿Cómo protegería las variables de entorno en AWS?

No subiría archivos .env al repositorio. Las variables sensibles, como credenciales, claves o URLs privadas, las configuraría directamente en el servicio de despliegue o las guardaría en herramientas como AWS Secrets Manager o Systems Manager Parameter Store. También limitaría los permisos para que solo la aplicación o las personas necesarias puedan leer esos valores.

Parte 4 - Uso de Inteligencia Artificial

Durante el desarrollo utilicé Codex como herramienta de apoyo. No lo usé para reemplazar el desarrollo completo, sino como una ayuda puntual para avanzar más rápido en algunas partes y revisar detalles.

En el backend lo usé poco. La mayor parte de la API la trabajé de forma manual para entender bien la estructura con Express, las rutas, los controladores, el servicio de tareas, el manejo de errores y las variables de entorno. Donde más me apoyé fue en el README, para dejar más claras las instrucciones de instalación, ejecución y endpoints disponibles.

En el frontend sí lo usé un poco más, principalmente para apoyar el diseño de la interfaz, porque esa parte puede tomar más tiempo entre estructura visual, estados de carga, mensajes de error, estilos responsivos y organización de componentes. Aun así, fui ajustando el resultado para que realmente consumiera la API que ya había creado en el backend.

Para verificar que el código era correcto, revisé que las rutas coincidieran entre frontend y backend. En el backend la API quedó bajo /api/tasks, y en el frontend configuré NEXT_PUBLIC_API_URL para que las funciones hicieran fetch correctamente a endpoints como /tasks, /tasks/:id y /tasks/:id/complete. También revisé manualmente el flujo de crear, listar, editar, completar y eliminar tareas, y ajusté cualquier detalle para que el frontend no quedara solo bonito, sino conectado a la API real.

Ejemplos de prompts utilizados

1. Prompt relacionado con frontend

Tengo una API REST de tareas hecha en Express con rutas para listar, crear, editar, completar y eliminar tareas. Ayúdame a mejorar una interfaz en Next.js para consumir esa API, mostrando estados de carga, errores, lista vacía y componentes reutilizables, pero manteniendo una estructura sencilla.

2. Prompt relacionado con README y documentación

Ayúdame a redactar un README claro para una API Node.js con Express que gestiona tareas en memoria. Incluye instalación, variables de entorno, comando para ejecutar el proyecto y ejemplos de endpoints como GET /api/tasks, POST /api/tasks y PATCH /api/tasks/:id/complete.

Verificación personal

Antes de dar por terminada la implementación, no tomé el resultado de la IA como definitivo. Comparé el código con los requisitos de la prueba, revisé que el backend respondiera con la estructura esperada y confirmé que el frontend estuviera llamando a la API correcta. Si alguna ruta o configuración no coincidía, la ajusté manualmente hasta que el flujo funcionara de punta a punta.

Me gusta usualmente el manejo de ramas locales para cualquier cosa eliminar la rama que es más sencillo o si son tareas simples con un descarte de cambios es suficiente.